

Ellenfél alapú támadások (FGSM, PGD)
implementációja
konvolúciós neurális hálózatokhoz
interaktív tesztkörnyezetben

Gabrity Gábor
Neptun kód: D9V09Z

2026. augusztus 23.

Nyilatkozat

Alulírott Gabrity Gábor (Neptun kód: D9V09Z), az Eötvös Loránd Tudományegyetem Informatikai Karának hallgatója kijelentem, hogy ezt a szakdolgozatot saját magam készítettem, és abban csak a megadott forrásokat használtam fel. Minden olyan részt, amelyet szó szerint vagy azonos tartalommal, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

A dolgozat bírálója által hátrányos értékeléssel nem járó exkluzív elektronikus hozzáférést a benyújtástól számított két évig nem kérem.

Budapest, 2026. augusztus 23.

Gabrity Gábor s.k.
hallgató aláírása

Kivonat

A mesterséges intelligencia rohamos fejlődésével párhuzamosan kritikus kérdéssé vált a modellek biztonsága és megbízhatósága. Kutatások kimutatták, hogy a modern konvolúciós neurális hálózati architektúrák sebezhetőek az ellenfél alapú támadásokkal szemben, mint például az FGSM (Fast Gradient Sign Method) és a PGD (Projected Gradient Descent). Ezek az algoritmusok a bemeneti adatot képesek olyan módon – az emberi szem számára szinte észrevétlenül – módosítani, hogy a nagy megbízhatóságú modelleket is megtévesztik.

Szakdolgozatom célja egy olyan interaktív tesztkörnyezet fejlesztése, amely grafikus felhasználói felületen keresztül teszi lehetővé ezen biztonsági rések vizuális elemzését. A projekt fókuszában az AI Red Teaming módszertan áll: a fejlesztő vagy biztonsági szakember a szoftver segítségével szisztematikusan „támadja” a választott képfelismerő modellt, hogy feltárja annak gyenge pontjait.

A keretrendszer több elismert támadási algoritmust implementál, köztük a gyors, egylépéses FGSM-et és a robusztusabb, iteratív PGD-t. A felhasználó számára lehetőség nyílik mind általános tévesztés (non-targeted), mind specifikus célosztályra irányuló (targeted) támadások végrehajtására, miközben interaktív módon, csúszkák segítségével szabályozhatja a perturbáció mértékét és az iterációk számát.

Az alkalmazás hármas vizuális megjelenítést alkalmaz: megjeleníti az eredeti képet, a generált zajt (perturbációt) és a végső, támadott képet, miközben mutatja a modell predikcióinak változását. A rendszer kiegészülhet egy modellvédelmi modulal is, amely különböző képfeldolgozási szűrők – például homályosítás – segítségével próbálja megsemmisíteni a beágyazott zajt, mérve ezzel a modell védekezőképességét.

Tartalomjegyzék

1. Bevezetés	7
1.1. Motiváció és a témaválasztás indoklása	7
1.2. A megoldandó feladat rövid, közérthető leírása	7
1.3. A dolgozat célja és hozzájárulásai	7
1.4. A dolgozat felépítése	7
2. Felhasználói dokumentáció	8
2.1. A megoldott probléma rövid megfogalmazása	8
2.2. A felhasznált módszerek rövid áttekintése	8
2.2.1. FGSM (Fast Gradient Sign Method) röviden	8
2.2.2. PGD (Projected Gradient Descent) röviden	8
2.2.3. Non-targeted és targeted támadási módok	8
2.3. Rendszerkövetelmények	8
2.4. Telepítés és első indítás	8
2.5. A grafikus felhasználói felület bemutatása	8
2.5.1. Modell kiválasztása és kép betöltése	8
2.5.2. Támadási paraméterek beállítása (ϵ , iterációszám, célosztály)	8
2.5.3. Az eredeti, a perturbáció és a támadott kép megjelenítése	8
2.5.4. Predikciók és konfidenciák leolvasása	8
2.6. Tipikus felhasználási forgatókönyvek	8
2.6.1. Non-targeted támadás végrehajtása	8
2.6.2. Targeted támadás egy adott célosztályra	8
2.6.3. Védekezési modul használata (szűrők alkalmazása)	8
2.7. Hibaüzenetek és gyakori problémák	8
3. Fejlesztői dokumentáció	9
3.1. A probléma részletes specifikációja	9
3.1.1. Funkcionális követelmények	9
3.1.2. Nem funkcionális követelmények	9
3.1.3. Használati esetek	9
3.2. Elméleti háttér és a használt fogalmak definíciója	9
3.2.1. Konvolúciós neurális hálózatok rövid áttekintése	9
3.2.2. Veszteségfüggvény, gradiens és backpropagation	9
3.2.3. Perturbáció, ℓ_∞ norma, ellenfél alapú példa	9
3.2.4. AI Red Teaming módszertan	9
3.3. Ellenfél alapú támadások áttekintése	9
3.4. FGSM (Fast Gradient Sign Method)	9
3.4.1. Matematikai háttér	9
3.4.2. Az algoritmus lépései	10
3.4.3. Célzott változat	10
3.4.4. Előnyök és korlátok	10
3.5. PGD (Projected Gradient Descent)	10
3.5.1. Matematikai háttér	10
3.5.2. Az algoritmus lépései	11
3.5.3. Célzott változat	11
3.5.4. Hiperparaméterek hatása	11
3.5.5. Előnyök és korlátok	11

3.6.	Kapcsolódó munkák és további támadási módszerek	12
3.6.1.	Iteratív FGSM (BIM) és Momentum-alapú változatok	12
3.6.2.	Carlini–Wagner és DeepFool támadások	12
3.6.3.	Transzferálhatóság és fekete dobozos támadások	12
3.7.	Védekezési stratégiák áttekintése	12
3.7.1.	Ellenfél alapú tanítás (adversarial training)	12
3.7.2.	Bemenet-előfeldolgozáson alapuló védekezés (szűrők, homályosítás)	12
3.7.3.	Detekció-alapú megközelítések	12
3.8.	A rendszer architektúrája	12
3.8.1.	Komponensek és modulfelbontás	12
3.8.2.	Adatfolyam a betöltéstől a támadott képig	12
3.8.3.	A GUI és a támadási motor kapcsolata	12
3.9.	A felhasznált technológiák	12
3.9.1.	Python és tudományos csomagok (NumPy, Matplotlib)	12
3.9.2.	Mélytanulási keretrendszer (PyTorch/TensorFlow)	12
3.9.3.	Grafikus felület (pl. Tkinter, PyQt vagy Streamlit)	12
3.10.	Az egyes komponensek megvalósítása	12
3.10.1.	Modellbetöltés és előfeldolgozás	12
3.10.2.	FGSM támadás implementációja	12
3.10.3.	PGD támadás implementációja	12
3.10.4.	Targeted és non-targeted mód kezelése	12
3.10.5.	Perturbáció megjelenítése és képnormalizálás	12
3.10.6.	Interaktív paraméterezés (csúszkák, célosztály választás)	12
3.10.7.	Védekezési modul (képfeldolgozási szűrők)	12
4.	Tesztelés	13
4.1.	Tesztelési terv	13
4.1.1.	Tesztelési célok és lefedettség	13
4.1.2.	Teszt adatkészletek és modellek kiválasztása	13
4.1.3.	Mérési metrikák (támadási sikerráta, ℓ_∞ eltérés, konfidencia)	13
4.2.	Egységtesztek	13
4.2.1.	FGSM modul tesztjei	13
4.2.2.	PGD modul tesztjei	13
4.2.3.	Segédfüggvények és képvágás tesztjei	13
4.3.	Integrációs tesztek	13
4.3.1.	GUI és támadási motor összekötése	13
4.3.2.	Modellbetöltés és teljes támadási folyamat	13
4.4.	Funkcionális tesztek és felhasználói validáció	13
4.5.	Teszteredmények és értelmezésük	13
4.5.1.	FGSM eredmények különböző ϵ értékekre	13
4.5.2.	PGD eredmények különböző iterációszámokra	13
4.5.3.	Targeted vs. non-targeted összehasonlítás	13
4.5.4.	A védekezési modul hatásának mérése	13
4.6.	A tesztelés korlátai	13
5.	Összefoglalás	14
5.1.	Elért eredmények	14
5.2.	Következtetések	14
5.3.	Továbbfejlesztési lehetőségek	14

1. Bevezetés

1.1. Motiváció és a témaválasztás indoklása

1.2. A megoldandó feladat rövid, közérthető leírása

1.3. A dolgozat célja és hozzájárulásai

1.4. A dolgozat felépítése

2. Felhasználói dokumentáció

2.1. A megoldott probléma rövid megfogalmazása

2.2. A felhasznált módszerek rövid áttekintése

2.2.1. FGSM (Fast Gradient Sign Method) röviden

2.2.2. PGD (Projected Gradient Descent) röviden

2.2.3. Non-targeted és targeted támadási módok

2.3. Rendszerkövetelmények

2.4. Telepítés és első indítás

2.5. A grafikus felhasználói felület bemutatása

2.5.1. Modell kiválasztása és kép betöltése

2.5.2. Támadási paraméterek beállítása (ϵ , iterációszám, célosztály)

2.5.3. Az eredeti, a perturbáció és a támadott kép megjelenítése

2.5.4. Predikciók és konfidenciák leolvasása

2.6. Tipikus felhasználási forgatókönyvek

2.6.1. Non-targeted támadás végrehajtása

2.6.2. Targeted támadás egy adott célosztályra

2.6.3. Védekezési modul használata (szűrők alkalmazása)

2.7. Hibaüzenetek és gyakori problémák

3. Fejlesztői dokumentáció

3.1. A probléma részletes specifikációja

3.1.1. Funkcionális követelmények

3.1.2. Nem funkcionális követelmények

3.1.3. Használati esetek

3.2. Elméleti háttér és a használt fogalmak definíciója

3.2.1. Konvolúciós neurális hálózatok rövid áttekintése

3.2.2. Veszteségfüggvény, gradiens és backpropagation

3.2.3. Perturbáció, ℓ_∞ norma, ellenfél alapú példa

3.2.4. AI Red Teaming módszertan

3.3. Ellenfél alapú támadások áttekintése

A mély neurális hálózatok – különösen a képfelismerésben alkalmazott konvolúciós architektúrák – rendkívül érzékenyek a bemeneti adatok apró, célzott módosításaira. Az ilyen módosításokat *ellenfél alapú perturbációknak* (adversarial perturbations) nevezzük, az ezeket előállító eljárásokat pedig *ellenfél alapú támadásoknak* (adversarial attacks) [1].

A támadások alapelve a következő: adott egy f osztályozó modell, egy x eredeti bemenet és annak helyes y címkéje. A cél egy olyan x_{adv} módosított bemenet előállítása, amelyre $f(x_{\text{adv}}) \neq y$, miközben a perturbáció $\delta = x_{\text{adv}} - x$ valamilyen norma szerint kicsi marad: $\|\delta\|_p \leq \epsilon$. A gyakorlatban leggyakrabban az ℓ_∞ normát alkalmazzák, amely a pixelenkénti maximális eltérést korlátozza.

A következő alfejezetekben a két legelterjedtebb gradiens alapú támadási módszert mutatom be részletesen.

3.4. FGSM (Fast Gradient Sign Method)

Az FGSM algoritmust Goodfellow és társai javasolták 2015-ben [1]. Az eljárás egyetlen lépésben állítja elő a perturbációt a veszteségfüggvény bemenetre vonatkozó gradiensének előjelét felhasználva.

3.4.1. Matematikai háttér

Legyen $J(\theta, x, y)$ a modell veszteségfüggvénye (például keresztentropia), ahol θ a hálózat tanult paraméterei, x a bemeneti kép és y az eredeti címke. Az FGSM a veszteségfüggvény x -re vonatkozó gradiensét számítja ki visszaterjesztéssel (backpropagation), majd a perturbációt a gradiens előjelének ϵ -szorosával képezi:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

A $\text{sign}(\cdot)$ függvény elemenként az előjelet adja vissza (-1 , 0 vagy $+1$), így a perturbáció minden pixel esetén pontosan $\pm\epsilon$ lesz. Ez biztosítja, hogy a módosítás az ℓ_∞ norma szerint pontosan ϵ nagyságú.

3.4.2. Az algoritmus lépései

1. Előreterjesztés (forward pass): a modell kiszámítja az x bemenethez tartozó predikciót és a veszteségfüggvény $J(\theta, x, y)$ értékét.
2. Visszaterjesztés (backward pass): kiszámítjuk a gradienst $\nabla_x J(\theta, x, y)$.
3. Perturbáció előállítása: $\delta = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$.
4. Támadott kép: $x_{\text{adv}} = \text{clip}(x + \delta, 0, 1)$, ahol a clip függvény biztosítja, hogy a pixelértékek a $[0, 1]$ tartományban maradjanak.

3.4.3. Célzott változat

Az alapváltozat (*non-targeted*) a helyes osztály veszteségét maximalizálja. A *targeted* változatban egy y_{target} célosztályt adunk meg, és a perturbációt úgy állítjuk elő, hogy a modell ebbe az osztályba sorolja a képet:

$$x_{\text{adv}} = x - \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y_{\text{target}}))$$

Az előjel megfordul, mert ebben az esetben a célosztályhoz tartozó veszteséget *minimalizáljuk*.

3.4.4. Előnyök és korlátok

Az FGSM fő előnye a sebessége: egyetlen gradiens-számítás elegendő. Hátránya, hogy egyetlen nagy lépést tesz, így kisebb ϵ értékek mellett gyakran kevésbé hatékony, mint az iteratív módszerek. Emellett a perturbáció iránya nem feltétlenül optimális, csupán a gradiens előjelét követi.

3.5. PGD (Projected Gradient Descent)

A PGD algoritmust Madry és társai vezették be 2018-ban [2] az FGSM iteratív általánosításaként. A módszert a szakirodalom gyakran a „legerősebb elsőrendű támadásnak” tekinti az ℓ_∞ korlátos perturbációk között.

3.5.1. Matematikai háttér

A PGD a következő iteratív frissítési szabályt alkalmazza, $t = 0, 1, \dots, T - 1$ lépésben:

$$x^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(x)} \left(x^{(t)} + \alpha \cdot \text{sign}(\nabla_x J(\theta, x^{(t)}, y)) \right)$$

ahol:

- α a lépésméret (step size), jellemzően $\alpha < \epsilon$,
- T az iterációk száma,
- $\Pi_{\mathcal{B}_\epsilon(x)}$ a projekciós operátor, amely a módosított mintát visszavetíti az eredeti x körüli ϵ sugarú ℓ_∞ gömbbe:

$$\Pi_{\mathcal{B}_\epsilon(x)}(z) = \text{clip}(z, x - \epsilon, x + \epsilon)$$

3.5.2. Az algoritmus lépései

1. Kezdeti pont megválasztása: $x^{(0)}$ lehet maga az eredeti x , vagy egy véletlen pont az $\mathcal{B}_\epsilon(x)$ környezetből (random restart).
2. Iteráció ($t = 0, \dots, T - 1$):
 - (a) Előreterjesztés és veszteségszámítás az aktuális $x^{(t)}$ pontra.
 - (b) Gradiens számítása: $g^{(t)} = \nabla_x J(\theta, x^{(t)}, y)$.
 - (c) Lépés a gradiens irányába: $\hat{x}^{(t+1)} = x^{(t)} + \alpha \cdot \text{sign}(g^{(t)})$.
 - (d) Projekció: $x^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(x)}(\hat{x}^{(t+1)})$.
 - (e) Pixelértékek vágása a $[0, 1]$ tartományba.
3. A végső támadott kép: $x_{\text{adv}} = x^{(T)}$.

3.5.3. Célzott változat

A targeted PGD a célzott FGSM-hez hasonlóan a célosztály veszteségét minimalizálja:

$$x^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(x)}\left(x^{(t)} - \alpha \cdot \text{sign}(\nabla_x J(\theta, x^{(t)}, y_{\text{target}}))\right)$$

3.5.4. Hiperparaméterek hatása

A PGD viselkedését három fő hiperparaméter határozza meg:

- ϵ : a megengedett maximális perturbáció. Nagyobb ϵ erősebb támadást tesz lehetővé, de a módosítás emberi szemmel is láthatóvá válhat.
- α : a lépésnagyság. Túl kis α lassú konvergenciát eredményez, túl nagy α pedig instabilitáshoz vezethet. Tipikus választás: $\alpha = \epsilon/T$ vagy $\alpha = 2.5 \cdot \epsilon/T$.
- T : az iterációk száma. Több iteráció általában erősebb támadást eredményez, de növeli a számítási költséget.

3.5.5. Előnyök és korlátok

A PGD az FGSM-nél lényegesen erősebb támadásokat képes generálni, mivel több kis lépésben közelíti a lokális optimumot. A random restart technikával kombinálva különösen hatékony. Hátránya, hogy T -szer annyi gradiens-számítást igényel, mint az FGSM, így nagyobb a számítási költsége.

3.6. Kapcsolódó munkák és további támadási módszerek

3.6.1. Iteratív FGSM (BIM) és Momentum-alapú változatok

3.6.2. Carlini–Wagner és DeepFool támadások

3.6.3. Transzferálhatóság és fekete dobozos támadások

3.7. Védekezési stratégiák áttekintése

3.7.1. Ellenfél alapú tanítás (adversarial training)

3.7.2. Bemenet-előfeldolgozáson alapuló védekezés (szűrők, homályosítás)

3.7.3. Detekció-alapú megközelítések

3.8. A rendszer architektúrája

3.8.1. Komponensek és modulfelbontás

3.8.2. Adatfolyam a betöltéstől a támadott képig

3.8.3. A GUI és a támadási motor kapcsolata

3.9. A felhasznált technológiák

3.9.1. Python és tudományos csomagok (NumPy, Matplotlib)

3.9.2. Mélytanulási keretrendszer (PyTorch/TensorFlow)

3.9.3. Grafikus felület (pl. Tkinter, PyQt vagy Streamlit)

3.10. Az egyes komponensek megvalósítása

3.10.1. Modellbetöltés és előfeldolgozás

3.10.2. FGSM támadás implementációja

3.10.3. PGD támadás implementációja

3.10.4. Targeted és non-targeted mód kezelése

3.10.5. Perturbáció megjelenítése és képnormalizálás

3.10.6. Interaktív paraméterezés (csúszkák, célosztály választás)

3.10.7. Védekezési modul (képfeldolgozási szűrők)

4. Tesztelés

4.1. Tesztelési terv

4.1.1. Tesztelési célok és lefedettség

4.1.2. Teszt adatkészletek és modellek kiválasztása

4.1.3. Mérési metrikák (támadási sikerráta, ℓ_∞ eltérés, konfidencia)

4.2. Egységtesztek

4.2.1. FGSM modul tesztjei

4.2.2. PGD modul tesztjei

4.2.3. Segédfüggvények és képvágás tesztjei

4.3. Integrációs tesztek

4.3.1. GUI és támadási motor összekötése

4.3.2. Modellbetöltés és teljes támadási folyamat

4.4. Funkcionális tesztek és felhasználói validáció

4.5. Teszteredmények és értelmezésük

4.5.1. FGSM eredmények különböző ϵ értékekre

4.5.2. PGD eredmények különböző iterációs számokra

4.5.3. Targeted vs. non-targeted összehasonlítás

4.5.4. A védekezési modul hatásának mérése

4.6. A tesztelés korlátai

5. Összefoglalás

5.1. Elért eredmények

5.2. Következtetések

5.3. Továbbfejlesztési lehetőségek

Mellékletek

Hivatkozások

- [1] Ian J. Goodfellow, Jonathon Shlens, Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. arXiv:1412.6572, 2015.
- [2] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, Adrian Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. arXiv:1706.06083, 2018.